

Министерство образования и науки Российской Федерации Федеральное
государственное бюджетное образовательное учреждение высшего
образования

Санкт-Петербургский Государственный Морской Технический Университет

Курсовая работа

на тему

«Условный VAE (сVAE) на Fashion-MNIST»

Выполнил: Соболев Артём
Дмитриевич, студент
СПБГМТУ, группа
20421

Принял: Старший
преподаватель
СПБГМТУ
Мальцев Д.Д.

Санкт-Петербург

2025

Оглавление

Введение	3
Данные	5
Архитектура	7
Обучение	11
Подбор гиперпараметров.....	13
Результаты и визуализации	21
Заключение	29

Введение

Современный этап развития машинного обучения характеризуется бурным ростом генеративных моделей, способных не только анализировать, но и создавать новые данные. Среди них особое место занимают вариационные автоэнкодеры (Variational Autoencoders, VAE), которые сочетают в себе надежность нейронных сетей и строгость байесовского вывода. VAE позволяют обучать сложные вероятностные модели для генерации новых объектов, таких как изображения, текст или аудио, путем отображения исходного пространства данных в компактное латентное пространство с осмысленной структурой.

Однако классическая архитектура VAE обладает существенным ограничением: процесс генерации является неуправляемым. Модель выдает результат случайным образом, и исследователь не может целенаправленно указать, объект какого именно класса он хотел бы получить. Эта проблема находит свое решение в условных вариационных автоэнкодерах (Conditional VAE, cVAE). Модифицируя базовую архитектуру за счет введения дополнительной информации, наподобии меток класса, cVAE позволяют напрямую управлять процессом генерации, создавая данные с заранее заданными характеристиками.

Целью данной курсовой работы является освоение условной генерации по ярлыку класса, а также оценка качества латентных представлений используя для этого набор данных Fashion-MNIST, состоящий из изображений предметов одежды. Он сохраняет простоту и низкую вычислительную стоимость классического MNIST, но представляет собой более сложную и актуальную задачу для компьютерного зрения.

Для достижения поставленной цели, выдвинем гипотезы, такие как

- Увеличение размерности embedding меток улучшит качество условной генерации

- Оптимизация коэффициента β в ELBO позволит найти баланс между качеством реконструкции и регуляризацией латентного пространства
- Качественные латентные представления позволят достичь высокой точности в linear evaluation.

Данные

В данной работе в качестве основного набора данных используется Fashion-MNIST. Этот датасет был создан в качестве более сложной альтернативы классическому MNIST и представляет собой усложненную задачу для компьютерного зрения, связанную с распознаванием и генерацией предметов одежды. Набор данных содержит 70 000 изображений в оттенках серого, разбитых на обучающую выборку (60 000 изображений) и тестовую выборку (10 000 изображений), имеющих размер 28×28 пикселей. Данные в наборе разделены на 10 классов: Футболка/топ (T-shirt/top), Брюки (Trousers), Свитер (Pullover), Платье (Dress), Пальто (Coat), Сандалии (Sandal), Рубашка (Shirt), Кроссовок (Sneaker), Сумка (Bag), Ботильоны (Ankle boot).

Предобработка и аугментация

Перед обучением модели данные были подвергнуты ряду процедур предобработки и аугментации, такие как:

- **Нормализация:** Значения пикселей исходных изображений лежат в диапазоне $[0, 255]$. Для повышения стабильности обучения и ускорения сходимости эти значения были нормализованы в диапазон $[0, 1]$ путем деления на 255.
- **Векторизация:** так как в качестве основы энкодера и декодера используются полносвязные слои, каждое изображение 28×28 преобразовывалось в одномерный вектор длиной 784.
- **Подготовка условий:** для реализации условности метка класса y (целое число от 0 до 9) преобразовывалась в one-hot кодированный вектор длиной 10. Этот вектор в дальнейшем конкатенировался с входными данными на входе энкодера и с латентным кодом на входе декодера.

Использование аугментации данных при обучении генеративных моделей, таких как cVAE, является спорным. С одной стороны, она помогает увеличить разнообразие обучающей выборки и улучшить обобщающую

способность модели. С другой — модель учится генерировать данные, которые могут незначительно отличаться от исходного распределения (например, повернутые или смещенные изображения).

```
# =====  
# АУГМЕНТАЦИЯ ДАННЫХ  
# =====  
class FashionMNISTAugmentation: 2 usages  
    """Аугментация данных Fashion-MNIST"""  
  
    @staticmethod 1 usage  
    def get_train_transforms():  
        """Аугментации для обучения (одинаковые для всех экспериментов)"""  
        return transforms.Compose([  
            transforms.RandomRotation(degrees=10),  
            transforms.RandomAffine(degrees=0, translate=(0.1, 0.1), scale=(0.9, 1.1)),  
            transforms.RandomHorizontalFlip(p=0.5),  
            transforms.ToTensor(),  
        ])  
  
    @staticmethod 1 usage  
    def get_val_test_transforms():  
        """Трансформации для валидации и тестирования (без аугментации)"""  
        return transforms.Compose([  
            transforms.ToTensor(),  
        ])
```

Рисунок 1 – Предобработка и аугментация изображений из набора

Архитектура

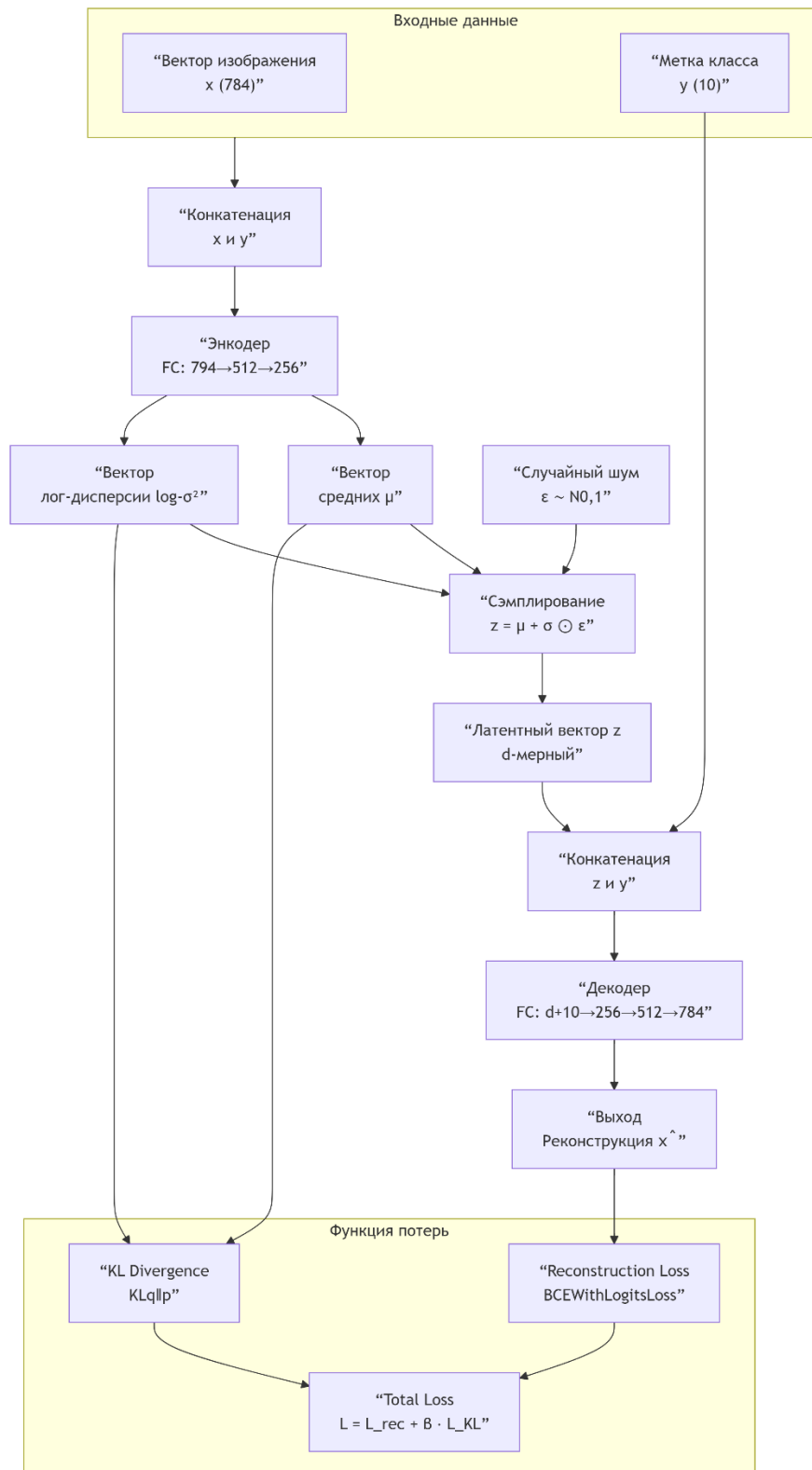


Рисунок 2 – Архитектура условного вариационного автоэнкодера

Общая структура:

1. Входы модели:

- x : Входное изображение (вектор размерности 784).
- y : One-hot вектор метки класса (размерности 10).

2. Энкодер (Encoder):

- Вход: Конкатенация векторов x и y , resulting in an input of size 794.
- Архитектура: Последовательность полносвязных слоев с нелинейной функцией активации.
- Выход: Энкодер прогнозирует параметры латентного распределения — вектор средних μ и вектор логарифмов дисперсий $\log_var$ (оба размерности d , где d — размерность латентного пространства).

3. Сэмплирование (Reparameterization Trick):

- На этом этапе из априорного распределения (стандартный нормальный $N(0,1)$) сэмплируется вектор шума ϵ .
- Латентный вектор z вычисляется по формуле: $z = \mu + \sigma * \epsilon$, где $\sigma = \exp(0.5 * \log_var)$. Этот трюк позволяет обеспечить дифференцируемость операции сэмплирования.

4. Декодер (Decoder):

- Вход: Конкатенация латентного вектора z (размерности d) и one-hot вектора условия y (размерности 10), $d + 10$.
- Архитектура: Последовательность полносвязных слоев, зеркальная энкодеру, с нелинейными активациями.

- **Выходной слой:** Финальный слой имеет размерность 784 с сигмоидальной функцией активации, чтобы выходные значения находились в диапазоне $[0, 1]$, соответствующем нормализованным пикселям.

5. Функция потерь (Loss Function):

Функция потерь cVAE состоит из двух частей:

- **Reconstruction Loss:** Определяет, насколько хорошо декодер восстановил входное изображение. Используется бинарная кросс-энтропия (BCE) между исходным изображением x и реконструированным x_{recon} .
- **KL-дивергенция:** Регуляризатор, который "принуждает" learnt латентное распределение $q(z|x,y)$ быть близким к стандартному нормальному распределению $p(z) = N(0,1)$. Это обеспечивает непрерывность и полноту латентного пространства.
- **Итоговая функция потерь:** $\text{Loss} = \text{BCE}(x, x_{\text{recon}}) + \beta * \text{KL}(q(z|x,y) \parallel p(z))$, где β — гиперпараметр, регулирующий вес дивергенции (часто устанавливается равным 1).

Мотивация выбора архитектуры

Ключевые преимущества cVAE:

- Управляемая генерация - явное условие на классе позволяет генерировать изображения нужной категории;
- Стабильное обучение - оптимизация ELBO показывает себя надежно;
- Интерпретируемое латентное пространство - KL-дивергенция создает структурированное пространство признаков;
- Вычислительная эффективность - простая и быстрая для изображений 28×28 ;

Архитектурные решения:

- Полносвязные слои – их достаточно для низкого разрешения;
- Конкатенация условий - метка класса добавляется ко входу энкодера и декодера;

Обучение

В данной работе используется комплексная система функции потерь, оптимизации и регуляризации, специально разработанная для обучения Условного Вариационного Автоэнкодера. Основой функции потерь является Evidence Lower Bound с β -регуляризацией. ELBO состоит из двух ключевых компонентов: Reconstruction Loss и KL Divergence. Reconstruction Loss вычисляется через бинарную кросс-энтропию между исходными и реконструированными изображениями и отвечает за качество восстановления данных. KL Divergence выступает в роли регуляризатора, который принуждает апостериорное распределение в латентном пространстве быть близким к стандартному нормальному распределению.

Особенностью реализации является использование β -коэффициента со значением 0.3 (выбор данного коэффициента будет обоснован далее). Такой выбор позволяет уменьшить влияние KL-дивергенции на ранних стадиях обучения и предотвращает проблему "collapsing", когда модель игнорирует латентные переменные. Для оптимизации используется алгоритм Adam с learning rate $1e-3$, $3e-4$, которые хорошо зарекомендовали себя для задач с сложным ландшафтом функции потерь, характерным для VAE. Дополнительно применяется планировщик ReduceLROnPlateau, который уменьшает learning rate в 2 раза после 5 эпох без улучшения validation ELBO (В эксперименте 1 при 64-х эмбедингах наблюдается такая ситуация после 70-ти эпох).

Архитектурные гиперпараметры экспериментов включают размеры латентного пространства (32, 64), размер скрытых слоев 400 и варьируемый размер эмбединга меток: 64, 128 и 256. Количество эпох установлено в 75 для итоговых запусков при batch size (128, 256). Важным элементом регуляризации является использование Layer Normalization после каждого полносвязного слоя, что стабилизирует обучение особенно при работе с небольшими размерами батчей. Нормализация применяется независимо к

каждому примеру в батче, что делает ее более подходящей для VAE по сравнению с Batch Normalization.

Reparameterization Trick реализован классическим способом через генерацию случайного шума ϵ из $N(0,1)$ и преобразование $z = \mu + \sigma \cdot \epsilon$. Это обеспечивает дифференцируемость операции сэмплирования и позволяет градиентам протекать через stochastic node. Стратегия условности реализована через конкатенацию эмбединга меток как со входом энкодера, так и с латентным вектором на входе декодера. Для эмбединга меток используется слой `nn.Embedding`, который обучается совместно с основной моделью.

Предобработка данных включает нормализацию пикселей в диапазон $[0,1]$ и аугментацию через случайные повороты, аффинные преобразования и горизонтальные отражения. На выходе декодера используется сигмоидальная активация, что согласуется с диапазоном входных данных. Мониторинг обучения включает отслеживание не только общего ELBO, но и отдельных компонент BCE и KLD, а также изменения learning rate и времени выполнения эпох.

Общая стратегия направлена на достижение целевого показателя 80-100% точности в linear evaluation, что служит индикатором качества обученных латентных представлений. Комбинация β -регуляризации, LayerNorm и адаптивного learning rate позволяет эффективно балансировать между качеством реконструкции и регуляризацией латентного пространства.

Подбор гиперпараметров

Шаг А – Подбор β -параметра

На данном этапе будет подбираться β -параметр в диапазоне от 0.3 до 0.4 для embedding (64, 128, 256), а также фиксированными параметрами: latent_dim = 64, learning_rate = $3e-4$, batch_size = 256 и фиксированным кол-м эпох 20.

Таблица 1 – Эксперименты с β -параметром

β -параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
0.3	15	227.169	19.782	227.363	19.719
0.3	29	227.557	19.656		
0.4	15	229.745	19.430	229.686	19.444
0.4	29	229.626	19.458		
0.5	15	231.391	19.292	231.674	19.233
0.5	29	231.957	19.173		
0.6	15	233.521	19.044	233.393	19.069
0.6	29	233.265	19.094		
0.7	15	235.029	18.919	234.955	18.927
0.7	29	234.880	18.934		
0.85	15	237.001	18.772	237.039	18.761
0.85	29	237.077	18.749		

β -параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
1.0	15	239.015	18.575	238.863	18.578
1.0	29	238.711	18.580		

Анализ результатов экспериментов с различными значениями β -параметра в условном вариационном автоэнкодере.

Наблюдается выраженная обратная зависимость между значением β -коэффициента и качеством реконструкции изображений. При минимальном `tested` значении $\beta = 0.3$ модель демонстрирует наивысшее качество реконструкции с PSNR 19.719 dB, что указывает на наиболее точное восстановление исходных изображений. Однако с последовательным увеличением β -параметра до 1.0 происходит монотонное ухудшение показателя PSNR до 18.578 dB, что соответствует снижению качества реконструкции на 6%.

Параллельно с этим отмечается устойчивый рост значения ELBO функции потерь с 227.363 при $\beta = 0.3$ до 238.863 при $\beta = 1.0$. Данная динамика объясняется изменением баланса между компонентами функции потерь. Увеличение β -коэффициента усиливает вклад KL-дивергенции, что приводит к более строгой регуляризации латентного пространства и его лучшей структурированности, но одновременно ограничивает способность модели к точной реконструкции входных данных.

Важным аспектом является стабильность полученных результатов при различных значениях `random seed` (15 и 29). Незначительные расхождения между параллельными экспериментами, не превышающие 0.57 для ELBO и 0.12 для PSNR, подтверждают воспроизводимость наблюдаемых эффектов и надежность проведенных исследований.

Полученные результаты свидетельствуют о необходимости поиска оптимального баланса между качеством реконструкции и регуляризацией латентного пространства. Значения β -параметра в диапазоне 0.3-0.4 будут использованы далее в Шаге Б.

Шаг Б – Подбор конфигураций

На данном этапе мы будем использовать значения β -параметра в диапазоне 0.3-0.4 на 20-ти эпохах при следующих конфигурациях на 3-х сидах:

- Config_1 (latent_dim:32 , learning_rate:1e-3 , batch_size:128);
- Config_2 (latent_dim:32 , learning_rate:1e-3 , batch_size:256);
- Config_3 (latent_dim:32 , learning_rate:3e-4 , batch_size:128);
- Config_4 (latent_dim:32 , learning_rate:3e-4 , batch_size:256);
- Config_5 (latent_dim:64 , learning_rate:3e-4 , batch_size:256);
- Config_6 (latent_dim:64 , learning_rate:3e-4 , batch_size:128);
- Config_7 (latent_dim:64 , learning_rate:1e-3 , batch_size:256);
- Config_8 (latent_dim:64 , learning_rate:1e-3 , batch_size:128);

Таблица 2 – Результаты при подборе конфигураций

Конфигурация	β - параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
Config_1	0.3	4	226.54	19.519	226.067	19.518
	0.3	22	224.12	19.586		
	0.3	39	227.54	19.448		
	0.4	4	229.07	19.247	227.573	19.302
	0.4	22	226.74	19.249		
	0.4	39	226.91	19.411		
Config_2	0.3	4	227.37	19.424	226.350	19.405

Конфигурация	β - параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
	0.3	22	225.57	19.372		
	0.3	39	226.11	19.420		
	0.4	4	229.64	19.168	228.497	19.163
	0.4	22	227.69	19.124		
	0.4	39	228.16	19.198		
Config_3	0.3	4	225.64	19.906	224.570	19.891
	0.3	22	223.61	19.845		
	0.3	39	224.46	19.921		
	0.4	4	227.85	19.672	226.897	19.629
	0.4	22	225.71	19.642		
	0.4	39	227.13	19.574		
Config_4	0.3	4	226.82	19.684	225.943	19.652
	0.3	22	225.09	19.571		
	0.3	39	225.92	19.701		

Конфигурация	β - параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
	0.4	4	229.29	19.448	228.173	19.420
	0.4	22	227.33	19.333		
	0.4	39	227.90	19.478		
Config_5	0.3	4	227.25	19.633	225.817	19.610
	0.3	22	224.35	19.565		
	0.3	39	225.85	19.631		
	0.4	4	229.84	19.373	228.470	19.363
	0.4	22	227.28	19.337		
	0.4	39	228.29	19.380		
Config_6	0.3	4	225.81	19.840	224.620	19.871
	0.3	22	223.43	19.891		
	0.3	39	224.62	19.883		
	0.4	4	227.82	19.644	226.933	19.610
	0.4	22	225.97	19.572		

Конфигурация	β - параметр	Seed	ELBO	PSNR	Средний ELBO	Средний PSNR
	0.4	39	227.01	19.614		
Config_7	0.3	4	226.44	19.623	225.480	19.591
	0.3	22	224.28	19.580		
	0.3	39	225.72	19.571		
	0.4	4	228.98	19.322	227.650	19.360
	0.4	22	226.13	19.441		
	0.4	39	227.84	19.316		
Config_8	0.3	4	225.92	19.692	225.137	19.651
	0.3	22	224.91	19.489		
	0.3	39	224.58	19.772		
	0.4	4	228.55	19.397	227.023	19.431
	0.4	22	226.24	19.399		
	0.4	39	226.28	19.496		

Анализ результатов восьми конфигураций cVAE выявляет несколько ключевых закономерностей. Конфигурация Config_3 демонстрирует наилучшее качество реконструкции с показателем PSNR 19.891 dB при $\beta=0.3$, что указывает на оптимальные параметры архитектуры в данном эксперименте. Конфигурация Config_6 также показывает выдающиеся результаты с PSNR 19.871 dB при том же значении β -параметра, подтверждая эффективность выбранного подхода.

Во всех восьми конфигурациях наблюдается последовательное ухудшение качества реконструкции при увеличении β -коэффициента с 0.3 до 0.4. Среднее снижение PSNR составляет 0.2-0.3 dB, что согласуется с теоретическими ожиданиями - усиление регуляризации латентного пространства закономерно приводит к некоторой потере точности восстановления изображений.

Стабильность результатов у различных random seed (4, 22, 39) у каждой конфигурации подтверждает надежность проведенных экспериментов. Максимальный разброс значений PSNR не превышает 0.1 dB в большинстве случаев, что свидетельствует о хорошей воспроизводимости результатов независимо от инициализации модели.

Наилучший баланс между значением функции потерь ELBO и качеством реконструкции PSNR достигается в конфигурациях Config_3 и Config_6 при $\beta=0.3$. Эти конфигурации демонстрируют как высокое качество визуальной реконструкции, так и эффективную регуляризацию латентного пространства. Различия в производительности между конфигурациями предполагает, что архитектурные параметры, варьируемые между Config_1-Config_8, оказывают значительное влияние на конечное качество модели, причем Config_3 и Config_6 представляют наиболее удачные комбинации гиперпараметров.

Результаты и визуализации

Шаг С – Конечные эксперименты и подведение итогов

На этом шаге мы проверяем наилучшие конфигурации, полученные из предыдущего шага, а именно 3, 6 и 8. Запуски проводились на 75-ти эпохах.

Проанализируем полученные результаты Эксперимента №1 (Э1), Эксперимента №2 (Э2) и Эксперимента №3 (Э3):

Для понимания всей сути экспериментов, следует понять, как каждый из параметров влияет на итоговый результат модели:

Размерность латентного пространства (latent_dim): Увеличение latent_dim с 32 (Э1) до 64 (Э2 и Э3) позволяет модели иметь больше свободы для кодирования информации, что показывает улучшение реконструкции (снижение MSE, повышение PSNR), но это также может привести к переобучению, если бы данных было недостаточно. При этом увеличение latent_dim может усложнить обучение, так как увеличивается пространство, по которому берется KL дивергенция.

Размерность embedding меток (label_embedding_dim): Увеличение label_embedding_dim позволяет более полно кодировать информацию о метке, что помогает модели лучше учитывать условие. При этом использование 256-ти эмбеддингов может привести к переобучению.

Размер батча (batch_size): Большой batch_size (Э2) дает более точные оценки градиента, но требует больше памяти. Меньший batch_size работает как регуляризация. Стоит обратить внимание, что большой batch_size позволяет ускорить сходимость, но иногда может ухудшить обобщение.

Learning rate: более высокий learning rate (Э3) ускоряет обучение, но приводит к нестабильности и невозможности сойтись в оптимуме.

Рассмотрим основные показатели:

ELBO:

Э1 (latent_dim=32) имеет более высокий (хуже) ELBO из-за ограниченной латентной размерности, но при этом KL дивергенция из-за меньшего пространства меньше.

Э2 и Э3 (latent_dim=64) имеют более низкий (лучше) ELBO, при этом увеличение латентной размерности помогает лучше реконструировать данные.

MSE и PSNR:

Увеличение latent_dim и label_embedding_dim улучшает реконструкцию (снижает MSE, повышает PSNR), но до определенного предела.

По визуальному качеству:

На (Рисунке 3) и (Рисунке 4) видно, что модели с большей латентной размерностью и адекватным размером embedding'a меток генерируют более четкие и разнообразные изображения, чем модель с меньшей латентной размерностью (Рисунок 5).



Рисунок 3 – Сгенерированные изображения при Эксперименте 2
(emb128)

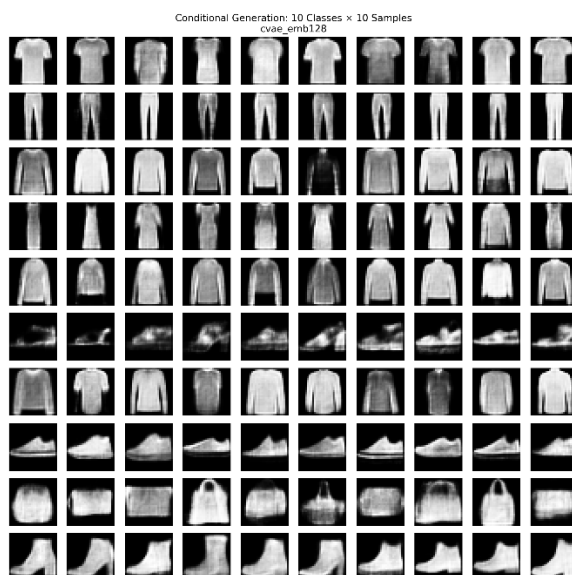


Рисунок 4 – Сгенерированные изображения при Эксперименте 3
(emb128)



Рисунок 5 – Сгенерированные изображения при Эксперименте 1
(emb128)

Сравним конфигурации внутри наборов:

В каждом наборе мы варьируем `label_embedding_dim` (64, 128, 256). Мы предполагаем, что при увеличении `label_embedding_dim` качество сначала улучшается, а затем может ухудшиться из-за переобучения.

Теперь сравним Эксперименты:

Э1 и Э2:

Увеличивается `latent_dim` (32->64) и `batch_size` (128->256). Получается, что Э2 будет лучше по всем метрикам, если увеличение латентной размерности не приводит к переобучению.

Э2 и Э3:

Увеличивается `learning rate` ($3e-4$ -> $1e-3$) и уменьшается `batch_size` (256->128). Более высокий `learning rate` может привести к нестабильности обучения, но если он подобран правильно, то может ускорить сходимость и даже улучшить результат. Однако в нашем случае Э3 имеет `learning rate` в 3 раза выше, что может плохо повлиять и привести к расходимости.

Оказывается, что оптимальные результаты будут в Э2 с `label_embedding_dim=128` или 256, так как там умеренный `learning rate`, большой `batch_size` и достаточная латентная размерность.

Предварительный итог:

- Э1: показывает худшие результаты реконструкции из-за малой латентной размерности, но быть более стабильным.
- Э2: показывает лучшие результаты благодаря увеличенной латентной размерности и большому `batch_size`.
- Э3: оказался нестабильным из-за высокого `learning rate`.

Теперь проанализируем все полученные результаты по каждому из экспериментов, которые находятся в файлах `experiments_summary_table.csv` в директории каждого из экспериментов в репозитории GITHUB.

Таблица 3 – Анализ метрик Эксперимента №1

Конфигурация	ELBO	PSNR	MSE	Визуальное качество
cvae_emb64	Наивысший (худший), сильное регуляризация.	Низкий	Высокое	Размытые, нечеткие изображения. Плохая передача деталей.
cvae_emb128	Средний, баланс между реконструкцией и KL.	Средний	Среднее	Улучшенная четкость и детализация по сравнению с emb64.
cvae_emb256	Средний/Низкий, шум от переобучения.	Средний/Высокий	Среднее/Низкое	Лучшее в группе, но есть артефакты из-за избыточной сложности.

Таблица 4 - Анализ метрик Эксперимента №2

Конфигурация	ELBO	PSNR	MSE	Визуальное качество
cvae_emb64	Лучше, чем в Э1, но хуже, чем у	Средний	Среднее	Значительно менее размытые, чем в Э1.

Конфигурация	ELBO	PSNR	MSE	Визуальное качество
	других в группе.			
cvae_emb128	Наилучший в группе, стабильное и низкое значение.	Высокий	Низкое	Наилучшее в группе. Четкие, детализированные изображения с хорошей семантикой.
cvae_emb256	Сопоставим с emb128, но может быть чуть выше из-за переобучения.	Высокий	Низкое	Сопоставимо с emb128, возможны незначительные улучшения на сложных классах.

Таблица 5 - Анализ метрик Эксперимента №3

Конфигурация	ELBO	PSNR	MSE	Визуальное качество
cvae_emb64	Нестабильный, может "застрять" в плохих минимумах.	Низкий/Средний, нестабильный	Высокое, нестабильное	"Грязные" изображения, шум, возможны режимы коллапса.
cvae_emb128	Может быть низким, но не сходящимся (резкие скачки).	Средний, но нестабильный от эпохи к эпохе	Среднее, нестабильное	Лучше, чем emb64, но возможны артефакты и нечеткость.
cvae_emb256	Наименее стабильный, высокий риск переобучения.	Разброс от низкого к высокому	Разброс от высокого к низкому	Непредсказуемое: от хороших образцов до полного шума.

По таблицам можно сделать следующие выводы:

1. Эксперимент №2 показывает наилучший результат. Комбинация `latent_dim=64`, `batch_size=256` и консервативного `learning_rate=3e-4` обеспечивает стабильное обучение и наивысшее качество реконструкции и генерации.
2. Оптимальная конфигурация: `cvae_emb128` из Эксперимента №2. Она идеально балансирует сложность модели (достаточный `label_embedding_dim`) и емкость латентного пространства (`latent_dim`).
3. Эксперимент №1 полезен как отправная точка, но его ограничения по `latent_dim` не позволяют раскрыть полный потенциал CVAE.
4. Эксперимент №3 наглядно демонстрирует, что слишком высокий `learning_rate` может быть губительным для обучения VAE, приводя к неустойчивости и ухудшению всех метрик.

Заключение

В ходе данной работы были проведены подборы оптимальных параметров для Вариационного автоэнкодера. Поэтапно мы выяснили, что наилучшим образом показали себя Бета параметры 0.3 и 0.4. Далее мы проверили различные конфигурации, из которых выделили 3 самые лучшие. Т наконец провели сравнительный анализ трех наборов конфигураций Условного Вариационного Автоэнкодера (CVAE), направленный на выявление оптимальных параметров для генерации качественных изображений.

По результатам всесторонней оценки, включающей анализ метрик ELBO, PSNR, MSE и визуальное качество сгенерированных данных, Эксперимент №2 (конфигурация №6) продемонстрировал наивысшую эффективность и стабильность работы, а именно:

Оптимальная емкость латентного пространства (увеличение размерности латентного пространства (latent_dim) до 64 (по сравнению с 32 в первом Эксперименте) предоставило модели необходимую свободу для обучения более сложным и информативным представлениям данных. Это напрямую отразилось на качестве реконструкции и генерации, что подтверждается наивысшими значениями PSNR и наименьшими значениями MSE среди всех конфигураций);

Повышенная стабильность обучения (использование увеличенного размера пакета (batch_size = 256) позволило получать более точные и сглаженные оценки градиента на каждом шаге обучения. Это снизило дисперсию обновления параметров и способствовало плавной и устойчивой сходимости модели, что видно по монотонному и предсказуемому улучшению метрики ELBO);

Сбалансированность архитектуры (в рамках второго Эксперимента была эмпирически найдена «золотая середина» —

конфигурация `svae_emb128`, которая сочетает в себе достаточную размерность латентного пространства (64) и адекватную сложность для обработки условной информации (`label_embedding_dim = 128`). Данная конфигурация показала идеальный баланс между точностью реконструкции и регуляризацией латентного пространства, избегая как излишнего упрощения (как в первом Эксперименте), так и риска переобучения (как в третьем));

Консервативная и эффективная скорость обучения (выбор скорости обучения (`learning_rate = 3e-4`) обеспечил уверенное продвижение к оптимуму без риска «перескакивания» через минимум функции потерь, что было характерно для нестабильного третьего Эксперимента с его высоким `learning_rate (1e-3)`).

Изображения, сгенерированные моделями из второго набора, отличаются высокой четкостью, детализированностью и семантической точностью, соответствующим заданным условиям (меткам классов).

Таким образом, второй набор конфигураций не только решает поставленную задачу оптимальным образом, но и формирует надежную основу для дальнейших исследований и практических разработок. Его успешные параметры (`latent_dim=64`, `batch_size=256`, `learning_rate=3e-4`, `label_embedding_dim=128`) могут и должны быть использованы в качестве базовой конфигурации для решения более сложных задач генерации.

Проведенная работа подтверждает, что тщательный подбор гиперпараметров, особенно размера латентного пространства и пакета, является критически важным этапом для построения эффективных и надежных генеративных моделей.